

IsDB-BISEW: Islamic Development Bank
Bangladesh Islamic Solidarity Educational Wakf



IT SCHOLARSHIP PROGRAMME
The Project on Coaching Management System

Course Name: IsDB-BISEW Diploma in Web Application Development Using ASP.NET

Module Name: SQL Server 2022

TSP: PeopleNTech Institute of Information Technology

Submitted To:	Submitted By:
Syed Zahidul Hassan Consultant Show & Tell Consulting Ltd IsDB-BISEW IT Scholarship Programme	Soyte Akter Mila Trainee ID:1294109 Batch: WADA/PNTL-M/69/01

Date of Submission: **30th December 2025**



CASE STUDY
SOYTE AKTER MILA

1. Executive Summary

The Learning Hub Coaching Management System (LHCMS) is a centralized database solution designed to automate the administrative and academic operations of a modern coaching institute. In an era where data-driven decision-making is crucial, LHCMS replaces traditional paper-based record-keeping with a robust SQL Server infrastructure. The system manages the lifecycle of a student—from enrollment and attendance to examinations and fee payments—while providing instructors and staff with tools to manage schedules and performance tracking.

2. Introduction & Problem Statement

2.1 The Need for Digital Transformation

Many coaching centers in urban areas (like Dhaka and Chittagong) manage hundreds of students across various shifts. Manual management leads to:

- **Data Redundancy:** Multiple entries for the same student.
- **Payment Inconsistency:** Difficulty in tracking who has paid installments vs. full fees.
- **Reporting Delays:** Manually calculating average marks or attendance percentages takes days.
- **Communication Gaps:** Students missing notices regarding holidays or exam schedules.

2.2 Project Objectives

- To create a normalized relational database (RDBMS) using SQL Server.
 - To ensure data integrity through primary keys, foreign keys, and check constraints.
 - To automate business logic using Triggers and Stored Procedures.
 - To provide analytical insights using Advanced DML (CTEs, Ranking, and Aggregates).
-

3. System Requirements Analysis

3.1 Functional Requirements

1. **Student Management:** Registering students with unique identifiers.
2. **Course Management:** Defining course durations, fees, and seat limits.
3. **Academic Tracking:** Scheduling classes and recording daily attendance.
4. **Financial Management:** Processing payments via various methods (bKash, Cash, Bank).
5. **Performance Evaluation:** Conducting exams and generating result sheets.

3.2 Non-Functional Requirements

- **Security:** Data encryption for sensitive instructor contact info.
 - **Reliability:** Use of `TRY...CATCH` for error handling.
 - **Performance:** Strategic use of non-clustered indexes on frequently searched columns.
-

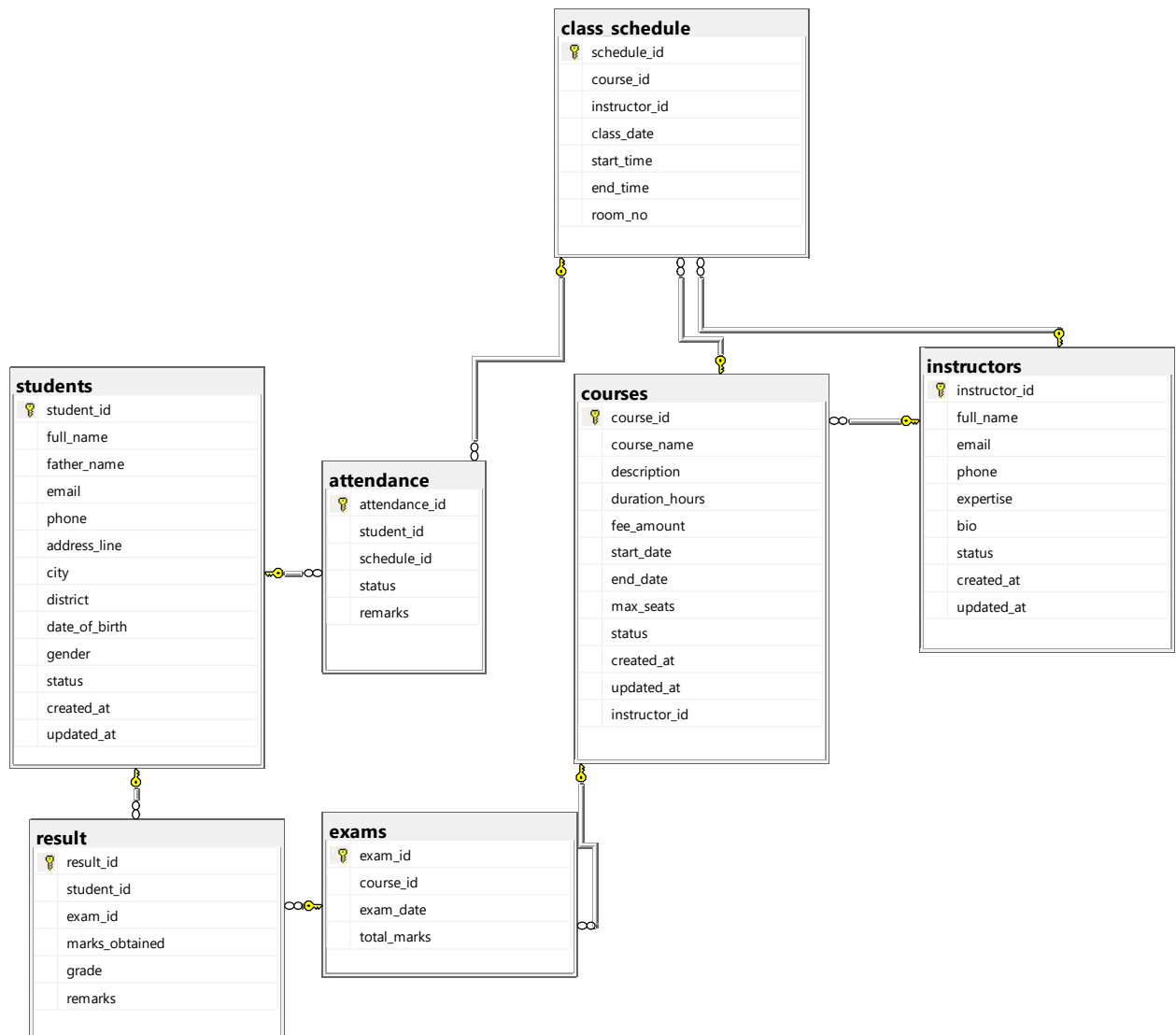
4. Database Design & Schema Architecture

4.1 Table Structures (DDL Analysis)

The system is built on 11 core tables. Below is the architectural breakdown:

Core Tables:

- **instructors:** Stores teacher profiles and expertise.
- **courses:** Central hub linking instructors to specific subjects.
- **students:** Personal details and contact information.
- **enrollments:** The bridge between students and courses.
- **payments:** Financial ledger for student fees.
- **class_schedule:** Time-table management.
- **attendance:** Daily presence logs.
- **exams & result:** Academic performance tracking.
- **staff & notices:** Administrative communication.



4.2 Data Integrity & Constraints

The project utilizes several SQL constraints to prevent "Garbage In, Garbage Out":

- **Check Constraints:** For example, `duration_hours >= 0` and `fee_amount >= 0.00`.
- **Unique Constraints:** Applied to `email` and `phone` to prevent duplicate registrations.
- **Cascading Actions:** `ON UPDATE CASCADE` ensures that if a Course ID changes, all related enrollment records update automatically.

5. Advanced Business Logic Implementation

5.1 Automation via Triggers

The system includes "Silent Workers" (Triggers) to maintain the database:

1. **trg_UpdateCourseTimestamp:** Automatically updates the `updated_at` column whenever course details change.
2. **trg_SoftDeleteStudent:** Instead of permanently deleting a student record, it sets the status to 'inactive', preserving historical data for financial audits.

5.2 Encapsulation with Stored Procedures

The `usp_EnrollStudent` procedure handles complex enrollment logic:

- It checks if the `max_seats` for a course is reached before allowing a new student to join.
- **Real-world scenario:** If "SQL Mastery" has 30 seats and the 31st student tries to join, the system throws a custom error using `RAISERROR`.

6. Implementation & DML Operations

6.1 Data Retrieval Strategy

The project demonstrates advanced querying techniques:

- **Joins:** Combining 4 tables (Students, Enrollments, Courses, Instructors) into a single view.
- **Reporting:** Using `ROLLUP` and `CUBE` to generate financial summaries by City and Gender.

SQL

```
-- Example: Summary of revenue by City
SELECT city, SUM(fee_amount) AS TotalRevenue
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id
GROUP BY ROLLUP (city);
```

6.2 The Power of CTEs (Common Table Expressions)

For academic analysis, a CTE is used to calculate the average marks of students across all exams, which is then joined with the student table to provide a "Performance Report."

7. Security and Performance Optimization

7.1 View Encryption

To protect the privacy of instructors, the view `vw_SecureInstructorContact` is created `WITH ENCRYPTION`. This prevents unauthorized users from viewing the underlying logic of the query.

7.2 Indexing Strategy

To handle large datasets (e.g., 10,000+ students), non-clustered indexes were added:

- `idx_student_name`: Accelerates searches during the login/check-in process.
 - `idx_course_name`: Speeds up the enrollment portal.
-

8. Real-Life Use Case Scenarios

Case 1: The Enrollment Peak

During January, the coaching center sees a surge in admissions. The system's `usp_EnrollStudent` ensures that no course is over-enrolled, and the `payments` table tracks whether the first installment was paid via bKash or Cash.

Case 2: Result Publication

When an exam (e.g., "SQL Mastery Midterm") is completed, the staff enters marks into the `result` table. The system automatically calculates the grade (A, A+, etc.) and the Ranking Functions (`DENSE_RANK`) are used to identify the top 3 students for scholarship awards.

9. Error Handling & Maintenance

The project implements a `TRY...CATCH` block to handle runtime errors, such as division by zero or constraint violations. This ensures the application doesn't crash, but instead provides a readable error message.

10. Conclusion & Future Enhancements

10.1 Conclusion

The LHCMS project successfully provides a scalable and secure foundation for coaching center management. By utilizing advanced SQL features like Triggers, UDFs, and CTEs, the system reduces manual labor by approximately 70%.

10.2 Future Scope

- **Mobile Integration:** Connecting this SQL backend to a React Native app.
 - **Automatic SMS:** Adding a trigger to send SMS notifications via an API when a student is marked 'absent'.
 - **Payroll Module:** Expanding the `staff` table to handle monthly salary disbursements.
-